

Programação linear - Exemplos

- O problema da mistura de petróleo
- Biblioteca Python

O Problema da mistura de petróleo

Refinaria processa vários tipos de petróleo para produzir gasolina de diferentes qualidades:

Superazul, Azul e Amarela.

- Os tipos de petróleo têm custos diferentes
- As gasolinas são produzidas com misturas diferentes

Quantidade disponível de petróleo

<i>Tipo de Petróleo</i>	<i>Quantidade Máxima Disponível (Barril/dia)</i>	<i>Custo por Barril/dia (R\$)</i>
1	3.500	19
2	2.200	24
3	4.200	20
4	1.800	27

Tabela criada por Goldbard e Luna, Otimização Combinatória e Programação Linear.

Percentuais para limites de qualidade das gasolinas.

<i>Tipo de Gasolina</i>	<i>Especificação</i>	<i>Preço de Venda R\$/Barril</i>
<i>Superazul</i>	Não mais que 30% de 1 Não menos que 40% de 2 Não mais que 50% de 3	35
<i>Azul</i>	Não mais que 30% de 1 Não menos que 10% de 2	28
<i>Amarela</i>	Não mais que 70% de 1	22

Tabela criada por Goldbard e Luna, Otimização Combinatória e Programação Linear.

Mistura de tipos
de petróleo para
obter a gasolina
azul

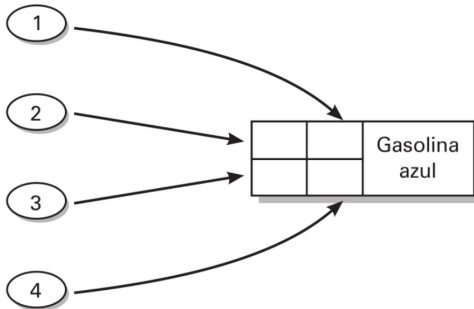


Figura criada por Goldband e Luna, Otimização
Combinatória e Programação Linear.

Solução

1. Escolha de variáveis

x_{ij} = número de barris de petróleo do tipo j
que serão destinados à produção da
gasolina i

$i = A, Z, S \equiv$ Amarela, Azul, Superazul

$j = 1, 2, 3, 4,$

$$x = (x_{ij})_{i,j}$$

Solução

2. Construir função objetivo

$\max f(x) =$ soma do lucro da venda dos diversos tipos de gasolina, abatidos os custos dos tipos de petróleo usados

$$\begin{aligned} = & 22(x_{A1} + x_{A2} + x_{A3} + x_{A4}) + 28(x_{z1} + x_{z2} + x_{z3} + x_{z4}) \\ & + 38(x_{s1} + x_{s2} + x_{s3} + x_{s4}) - 19(x_{A1} + x_{z1} + x_{s1}) \\ & - 24(x_{A2} + x_{z2} + x_{s2}) - 20(x_{A3} + x_{z3} + x_{s3}) \\ & - 27(x_{A4} + x_{z4} + x_{s4}) \end{aligned}$$

Solução

3. Formulações das restrições.

a) Quantidade de petróleo disponível

$$\text{Tipo 1: } x_{A1} + x_{Z1} + x_{S1} \leq 3500$$

$$\text{Tipo 2: } x_{A2} + x_{Z2} + x_{S2} \leq 2200$$

$$\text{Tipo 3: } x_{A3} + x_{Z3} + x_{S3} \leq 4200$$

$$\text{Tipo 4: } x_{A4} + x_{Z4} + x_{S4} \leq 1800$$

Solução

3. Formulações das restrições.

b) Especificações da mistura

Superazul $x_{s1} \leq 0.3 (x_{s1} + x_{s2} + x_{s3} + x_{s4})$

$$x_{s2} \geq 0.4 (x_{s1} + x_{s2} + x_{s3} + x_{s4})$$

$$x_{s3} \leq 0.5 (x_{s1} + x_{s2} + x_{s3} + x_{s4})$$

Azul $x_{z1} \leq 0.3 (x_{z1} + x_{z2} + x_{z3} + x_{z4})$

$$x_{z1} \geq 0.1 (x_{z1} + x_{z2} + x_{z3} + x_{z4})$$

Amarula $x_{A1} \leq 0.7 (x_{A1} + x_{A2} + x_{A3} + x_{A4})$

Solução

3. Formulações das restrições.

c) Restrições de não-negatividade

$$x_{A1} \geq 0, x_{A2} \geq 0, x_{A3} \geq 0, x_{A4} \geq 0$$

$$x_{z1} \geq 0, x_{z2} \geq 0, x_{z3} \geq 0, x_{z4} \geq 0$$

$$x_{s1} \geq 0, x_{s2} \geq 0, x_{s3} \geq 0, x_{s4} \geq 0$$

Exercício 1.

Usando uma "calculadora simplex" ou biblioteca em Python, resolver o problema acima.

Veja links na página do curso ou exercício a seguir.

Exercício 2

Usando Python, resolva

$$\max 3x + 2y$$

sujeito a

$$\begin{cases} x + y \leq 4 \\ 2x + y \leq 5 \\ x, y \geq 0 \end{cases}$$

Veja o código a seguir.

```
from scipy.optimize import linprog

# Função Objetivo
c = [-3, -2]

# Vínculos
A = [
    [1, 1],
    [2, 1]
]
b = [4, 5]

# Variáveis não-negativas  $x \geq 0$ ,  $y \geq 0$ 
bounds = [(0, None), (0, None)]

# Solução
result = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='simplex')

# Saída
print("Valores ótimos:", result.x)
print("Valor máximo:", -result.fun)
```

```
: Valores ótimos: [1. 3.]
  Valor máximo: 9.0
```

```
/tmp/ipykernel_414/1534885413.py:17: DeprecationWarning:
`method='simplex'` is deprecated and will be removed in SciPy 1.11.0.
Please use one of the HiGHS solvers (e.g. `method='highs'`) in new code.
    result = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='simplex')
```

O algoritmo simplex será removido em uma versão futura da biblioteca. A sugestão é usar a nova biblioteca `highs` (<https://highs.dev/>)

```
from scipy.optimize import linprog

# Função Objetivo
c = [-3, -2]

# Vínculos
A = [
    [1, 1],
    [2, 1]
]
b = [4, 5]

# Variáveis não-negativas x >= 0, y >= 0
bounds = [(0, None), (0, None)]

# Solução
result = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')

# Saída
print("Valores ótimos:", result.x)
print("Valor máximo:", -result.fun)
```

```
Valores ótimos: [1. 3.]
Valor máximo: 9.0
```